

Programmierung 2 - Sommersemester 2026

Prof. Dr. Markus Esch

Übung Nr. 1 Abgabe KW 16

Hamurabi ist eine textbasierte Wirtschaftssimulation aus dem Jahr 1968, die ursprünglich in FOCAL geschrieben und später nach BASIC portiert wurde.

Wikipedia beschreibt den Spielablauf wie folgt:

In Hamurabi verwaltet der Spieler als König einen antiken Stadtstaat. Ziel des Spiels ist es, innerhalb von zehn Runden möglichst wenige Hungertote zu verursachen und das Gebiet des Staates auszuweiten. Dabei ist jede Runde neu zu entscheiden, wie viele Morgen Land gekauft oder verkauft werden und wie viele Scheffel Getreide an die Bevölkerung als Nahrung ausgegeben oder als Saat für die kommende Ernte verwendet werden sollen.

Spielbeeinflussende Zufallsfaktoren sind ein variierender Ernteertrag pro Morgen und ein sich verändernder Landpreis. Als weitere Zufallsereignisse können eine Landplage auftreten, die die Bevölkerung dezimiert, und Ratten, die eine bestimmte Menge des gelagerten Getreides auffressen.

Sollte der Spieler nicht vor Ablauf der zehnten Runde von seiner Bevölkerung abgesetzt oder ermordet worden sein, erhält er am Ende eine leicht humoristisch anmutende Beurteilung seiner Leistung (Quelle: <https://de.wikipedia.org/wiki/Hamurabi>).

Heute gibt es verschiedene Portierungen des Spiels, so dass es auch im Browser gespielt werden kann, unter anderem hier:

<https://adventuron.itch.io/hamurabi>

Probieren Sie das Spiel einmal aus, das Verständnis des Spielablaufs wird Ihnen die Umsetzung sicherlich erleichtern.

Im *Codeopolis*-Projekt wollen wir eine Java-Version des Spiels entwickeln. Wir wollen uns aber nicht mit dem ursprünglichen Funktionsumfang von Hamurabi begnügen, sondern werden das Spiel im Laufe der Zeit deutlich erweitern. *Codeopolis* ist ein Kofferwort, das die Begriffe *Code* und *Metropolis* kombiniert. Es verbindet die Welt der Programmierung mit dem Spielziel, eine Stadt zu einer blühenden Metropole zu entwickeln.

Am Ende des ersten Semesters haben Sie für die vorlesungsfreie Zeit eine Reihe von freiwilligen Übungen zum Codeopolis-Projekt erhalten. Einige von Ihnen haben diese Aufgaben vielleicht bearbeitet. In

den Übungen wurden alle Themen von Programmierung 1 wiederholt. In Programmierung 2 werden wir an dieser Stelle fortfahren und die im Laufe des Semesters neu erlernten Programmier Techniken zur Erweiterung des Spiels nutzen. Dies soll Ihnen die Möglichkeit geben, die im Studium erlernten Programmier Techniken im Kontext eines größeren Programmierprojektes anzuwenden und so den Gesamtzusammenhang zu erkennen.

1. Aufgabe

In diesem Semester werden wir das Codeopolis-Projekt im Rahmen der Übungen fortführen. Damit alle auf einem gleichen Stand beginnen können, steht Ihnen in Moodle eine Basisimplementierung für das Modul Programmierung 2 zur Verfügung.

In einem ersten Schritt sollen Sie die Inhalte aus Programmierung 1 wiederholen und sich die Codeopolis-Implementierung in Erinnerung rufen. Gehen Sie dabei wie folgt vor:

- Importieren Sie die Basisimplementierung in IntelliJ IDEA (oder eine andere IDE ihrer Wahl) und führen Sie das Programm aus.
- Machen Sie sich mit dem Code vertraut und verstehen Sie die Struktur der Implementierung sowie die Funktion der einzelnen Klassen und deren Zusammenhang. Im Code-Review zu diesem Übungsblatt müssen Sie in der Lage sein dies zu erklären. Unten finden Sie eine Reihe von Fragen zur Implementierung. Stellen Sie sicher, dass Sie hinreichendes Verständnis des Codes haben, um diese und ähnliche Fragen zu beantworten.
- Im Peer-Review sollen sich die beiden Coding-Teams abwechseln zufällig ausgewählte Fragen stellen und erklären.

Fragen zur Basisimplementierung:

- Beschreiben Sie die Gesamtarchitektur des Spiels Codeopolis. Wie sind die Klassen innerhalb des Pakets `de.htwsaar.esch.Codeopolis.DomainModel` organisiert?
- Erklären Sie die Rolle der `UserInterface`-Schnittstelle. Wie entkoppelt sie die Spiellogik von der Benutzeroberfläche?
- Diskutieren Sie, wie die `Grain`-Klasse und ihre Unterklassen wie `Wheat`, `Corn` und `Rice` verschiedene Pflanzentypen darstellen. Wie werden spezifische Verhaltensweisen wie Dürreanpassungen implementiert?
- Erklären Sie die Beziehung zwischen den Klassen `City` und `CityState`. Zu welchem Zweck verwendet die `City`-Klasse `CityState`?
- Welche Konfigurationsoptionen bietet die `GameConfig`-Klasse? Wie beeinflussen diese Einstellungen das Gameplay in Codeopolis?
- Beschreiben Sie, wie die `Depot`- und `Silo`-Klassen die Lagerung von Getreide verwalten. Welche Mechanismen gibt es, um Kapazität und Getreideverfall zu handhaben?
- Analysieren Sie die `TurnResult`-Klasse. Welche Informationen fasst sie zusammen, und wie werden diese Informationen nach jedem Spielzug verwendet?
- Diskutieren Sie den Spielablauf innerhalb der `Game`-Klasse. Wie werden die verschiedenen Phasen eines Spielzugs gehandhabt, und welche Bedingungen führen zum Spielende?
- Erklären Sie an welchen Stellen und zu welchem Zweck Exceptions verwendet werden. Erklären Sie die unterschiedlichen Exceptions.

- Wie werden Harvest-Objekte erzeugt?
- Können Sie die Vererbungshierarchie für die Klassen `Grain`, `WinterGrain`, `SpringGrain`, `Wheat`, `Corn` usw. erläutern? Wie wird Polymorphie in diesem Kontext genutzt?
- Wie interagiert die Klasse `Game` mit der `UserInterface`-Schnittstelle, um Spielereignisse zu verarbeiten und Benutzeraktionen zu steuern?
- Wie wird in `Depot` und `Silo` entschieden, ob Getreide hinzugefügt oder entfernt werden kann, und wie wird der freie Speicherplatz verwaltet?
- Wie ist das Speichern und Laden des Spielstandes implementiert? Welches Java-Interface wird verwendet?
- Erklären Sie den Kontrollfluss von einer Benutzereingabe über die Ausführung der Aktion bis zur Anzeige des Ergebnisses. Welche Klassen und Methoden werden aufgerufen und in welcher Reihenfolge?
- Erklären Sie die Implementierung der Methoden `buy`, `sell`, `feed` und `plant`. Was machen die Methoden und wie werden falsche Eingaben behandelt?
- Erklären Sie die `City.runTurn()`-Methode. Was macht die Methode und wann wird sie aufgerufen?
- Wie interagieren die Klassen `Depot` und `Silo` miteinander, und was ist der Unterschied zwischen diesen beiden in Bezug auf die Speicherung von Ressourcen?
- Wie werden Daten und Operationen innerhalb der Klassen `Depot` und `Silo`, um sicherzustellen, dass nur gültige Aktionen durchgeführt werden können?
- Welche Methoden besitzt die `Depot`-Klasse und was machen die einzelnen Methoden?
- Beschreiben Sie, wie die Methode `store` in der `Depot`-Klasse funktioniert. Wie wird entschieden, in welchem `Silo` die Ernte gespeichert wird? Welchen Zweck hat dabei die Methode `defragment`?
- Wie ist das Verrotten von Getreide implementiert?
- Wie wird das Enum `Game.GrainType` verwendet, um die Typen von Getreide in den Klassen wie `Harvest` und `Silo` zu unterscheiden?
- Geben Sie ein Beispiel, wie das Enum `Game.GrainType` in einer Methode der `City`-Klasse verwendet wird.
- Erklären Sie, wie das Enum `Game.GrainType` bei der Implementierung der Lagerverwaltung im `Depot` verwendet wird.
- Wie werden Eingaben des Benutzers in der `Dialog`-Klasse verarbeitet und validiert?

2. Aufgabe (Codeopolis)

Es soll nun eine kleine Verbesserung am Code vorgenommen werden, die die Funktionalität nicht verändert, aber den Code sauberer und lesbarer macht.

In der Methode `City.runTurn()` wird zufällig entschieden, ob eine Dürre herrscht und ob Krankheiten oder Schädlinge auftreten. Integrieren Sie diese Werte in die Klasse `Conditions`. Beim Aufruf der Methode `Conditions.generateRandomConditions` sollen die Werte zufällig gesetzt werden und über Getter-Methoden abrufbar sein.

Lernziele:

Nach der Bearbeitung des Übungsblattes sollten Sie ...

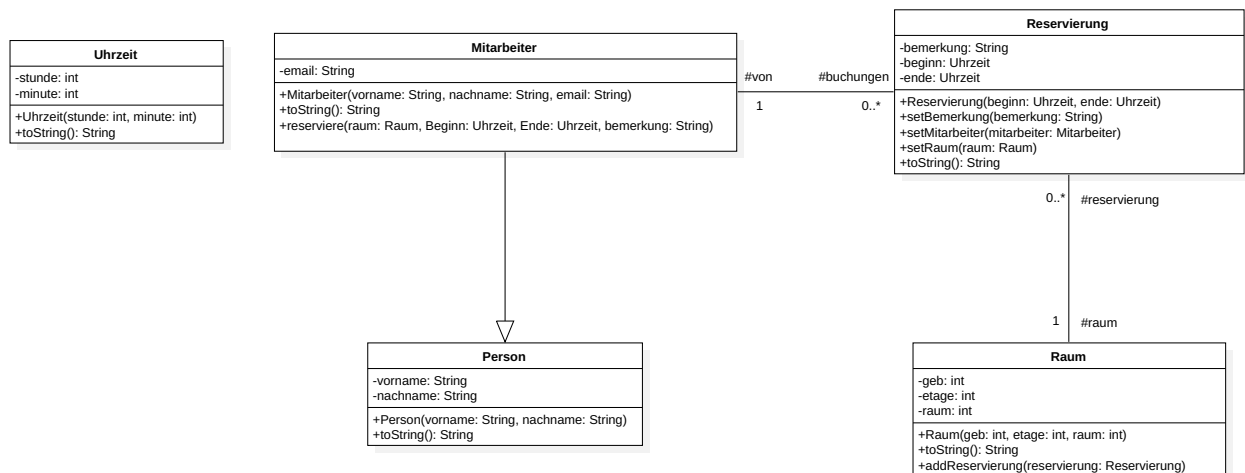
- ... die Inhalte aus Programmierung 1 erfolgreich rekapituliert haben.
- ... in der Lage sein, die Struktur der Baseline-Lösung zu erläutern.
- ... erklären können, was die einzelnen Klassen und Methoden der Baseline-Lösung machen und wie sie zusammenhängen.
- ... den Kontrollfluss erklären können.
- ... erkannt haben, wie die in Programmierung 1 erlernten Programmiertechniken angewendet wurden.
- ... in der Lage sein, eine kleine Anpassung des Codes vorzunehmen.
- ... den Code so gut verstanden haben, dass sie ihn in den nächsten Übungen weiterentwickeln können.

Zusatzaufgaben

Die folgenden Aufgaben sind Zusatzaufgaben und nicht Teil des Peer Reviews! Sie können die Aufgaben freiwillig bearbeiten, um die Inhalte weiter zu üben. Selbstverständlich können Sie auch Unterstützung und Beratung zu den Zusatzaufgaben erhalten.

1. Aufgabe

Gegeben sei folgendes UML Diagramm.



Ergänzend sind Ihnen folgende Hinweise für die Implementierung gegeben. Eine Uhrzeit soll wie folgt angelegt werden können:

```
Uhrzeit u = new Uhrzeit(12, 54);
```

Mittels `System.out.println(u)` soll eine Uhrzeit wie folgt ausgegeben werden können:

```
12:54 Uhr
```

Ein Mitarbeiter soll wie folgt angelegt

```
Mitarbeiter m = new Mitarbeiter("Max", "Mustermann", "max@htwsaar.de");
```

und mittels `System.out.println(m)` wie folgt ausgegeben werden können:

```
Max Mustermann (max@htwsaar.de)
```

Ein Raum soll wie folgt angelegt

```
Raum r = new Raum(18, 0, 1);
```

und mittels `System.out.println(r)` wie folgt ausgegeben werden können:

```
Raum 18-0.1
```

Mitarbeiter können Räume wie folgt reservieren. Sind beispielsweise die folgenden Mitarbeiter und Räume gegeben

```
Mitarbeiter m1 = new Mitarbeiter("Max", "Mustermann", "max@htwsaar.de");
```

```
Mitarbeiter m2 = new Mitarbeiter("John", "Doe", "john@htwsaar.de");
```

```
Raum r1 = new Raum(18,0,1);
```

```
Raum r2 = new Raum(2,1,9);
```

```
Raum r3 = new Raum(2,1,10);
```

und werden zusätzlich die folgenden Anweisungen ausgeführt

```
m1.reserviere(r1, new Uhrzeit(12, 30), new Uhrzeit(14, 30), "VOOP");
m1.reserviere(r2, new Uhrzeit(14, 30), new Uhrzeit(16, 30), "WebTech");
m2.reserviere(r2, new Uhrzeit(12, 30), new Uhrzeit(13, 30), "Prog II");
m2.reserviere(r3, new Uhrzeit(9, 30), new Uhrzeit(11,30), "ITM");
```

dann soll die Raumausgabe von `System.out.println(r1)` nun zusätzliche Reservierungsinformationen beinhalten:

Raum 18-0.1

gebucht von Max Mustermann (max@htwsaar.de) von 12:30 Uhr bis 14:30 für VOOP

Auch die Ausgabe von `System.out.println(r2)` soll dann diese Ausgabe erzeugen:

Raum 2-1.9

gebucht von Max Mustermann (max@htwsaar.de) von 14:30 Uhr bis 16:30 für WebTech

gebucht von John Doe (john@htwsaar.de) von 12:30 Uhr bis 13:30 Uhr für Prog II

Gleiches gilt für die Ausgabe von `System.out.println(r3)`:

Raum 2-1.10

gebucht von John Doe (john@htwsaar.de) von 9:30 Uhr bis 11:30 Uhr für ITM

Setzen Sie das Klassendiagramm unter Berücksichtigung der Implementierungshinweise sowie der folgenden ergänzenden Hinweise bitte in Java Code um.

- Beachten Sie bitte, dass Sie offenbar über Assoziationen von Räumen auf Mitarbeiter schließen können müssen. Diese Beziehungen müssen also mittels geeigneter Methoden verwaltet werden.
- Sie müssen keine Doppelbelegungen erkennen oder zeitlichen Sortierungen bei den Ausgaben vornehmen.
- Sie müssen ebenso keine Reservierungsrücknahmen, Umbuchungen oder ähnliches berücksichtigen.
- Der Index bei der Funktion `getReservierung(int index)` beginnt bei 0.

2. Aufgabe

Implementieren Sie ein Dialog-Programm zum Testen der in Aufgabe 1 implementierten Klassen.